



StoreFlow

Superstore & Grocery Store Management System — Complete Project Documentation

Web Application · Mobile Application · Backend API

Prepared as a complete reference: introduction, architecture, features,
setup steps, commands, hosting plan, and future roadmap.

Table of Contents

1. Introduction & Project Purpose
2. Technology Stack
3. System Architecture
4. User Roles & Permissions
5. Complete Feature List (Base → Intermediate → Advanced)
6. Local Development Setup (Step by Step)
7. Complete Command Reference
8. Building & Distributing the Mobile App (EAS Build)
9. Hosting & Deployment Plan
10. SMS & WhatsApp Messaging Solution
11. Security Notes
12. Future Roadmap
13. Appendix — Full Terminal Command List

1. Introduction & Project Purpose

StoreFlow is a management system built for superstores and grocery stores. It replaces manual, paper-based (or scattered spreadsheet) processes with a single connected system that covers inventory, order taking, worker coordination, payments, and reporting — usable from a computer (web app) or a phone (mobile app).

The system is **multi-tenant**, meaning one deployment of StoreFlow can serve many separate businesses at the same time. Each business ("tenant") only ever sees its own data — its own items, orders, staff, and customers — even though everyone is using the same backend and database.

1.1 Who is it for?

- **Store Owners** — who want to manage inventory, staff, and see sales reports without hiring a developer for every change.
- **Counter Staff** — who take customer orders at the counter/till.
- **Store Workers** — who physically collect items from the shelves to fulfill an order.
- **Super Admin (platform owner)** — the person who runs StoreFlow itself as a service to multiple store businesses (i.e. the developer/business selling this system to multiple clients).

1.2 The Three Parts of the Project

Part	What it is	Who uses it
Backend	A Laravel (PHP) REST API — the "brain" of the system. Stores all data and enforces all business rules and permissions.	Not used directly by people — both the Web App and Mobile App talk to it.
Frontend (Web App)	A React single-page application used from a browser on a desktop/laptop.	Owners, Counter Staff, Store Workers, Super Admin.
Mobile App	A React Native (Expo) Android/iOS app with the same features as the web app.	Same roles, used on a phone/tablet.

Both the Web App and the Mobile App are just two different "front doors" to the exact same Backend API — so a feature added to the backend is instantly available to both, and data stays perfectly in sync between web and mobile.

2. Technology Stack

2.1 Backend

Technology	Version	Purpose
PHP	^8.3	Programming language the backend runs on
Laravel	^13.8	The web framework — routing, database, validation, etc.
SQLite	—	Database used during development (a single file, no server needed)
Laravel Sanctum	^4.0	Login/authentication using secure tokens (no cookies) — same tokens work for web and mobile
Laravel Reverb	^1.11	Real-time WebSocket server — instantly pushes new orders to a Store Worker's screen on the web app, no refresh needed
Maatwebsite/Excel	^3.1	Reads uploaded Excel/CSV files to bulk-import inventory items
Twilio SDK	^8.11	Sends real SMS text messages (used for due-payment reminders)

2.2 Frontend (Web App)

Technology	Version	Purpose
React	^19.2	UI library — builds the interface out of reusable components
Vite	^8.1	Fast build tool / dev server for the React app
React Router	^7.18	Page navigation without full page reloads
Laravel Echo + Pusher-js	^2.4 / ^8.5	Connects the browser to Reverb's WebSocket server for real-time order updates
Recharts	^3.10	Draws the sales/report charts (line, bar, donut)
lucide-react	^1.25	Icon set used throughout the interface

2.3 Mobile App

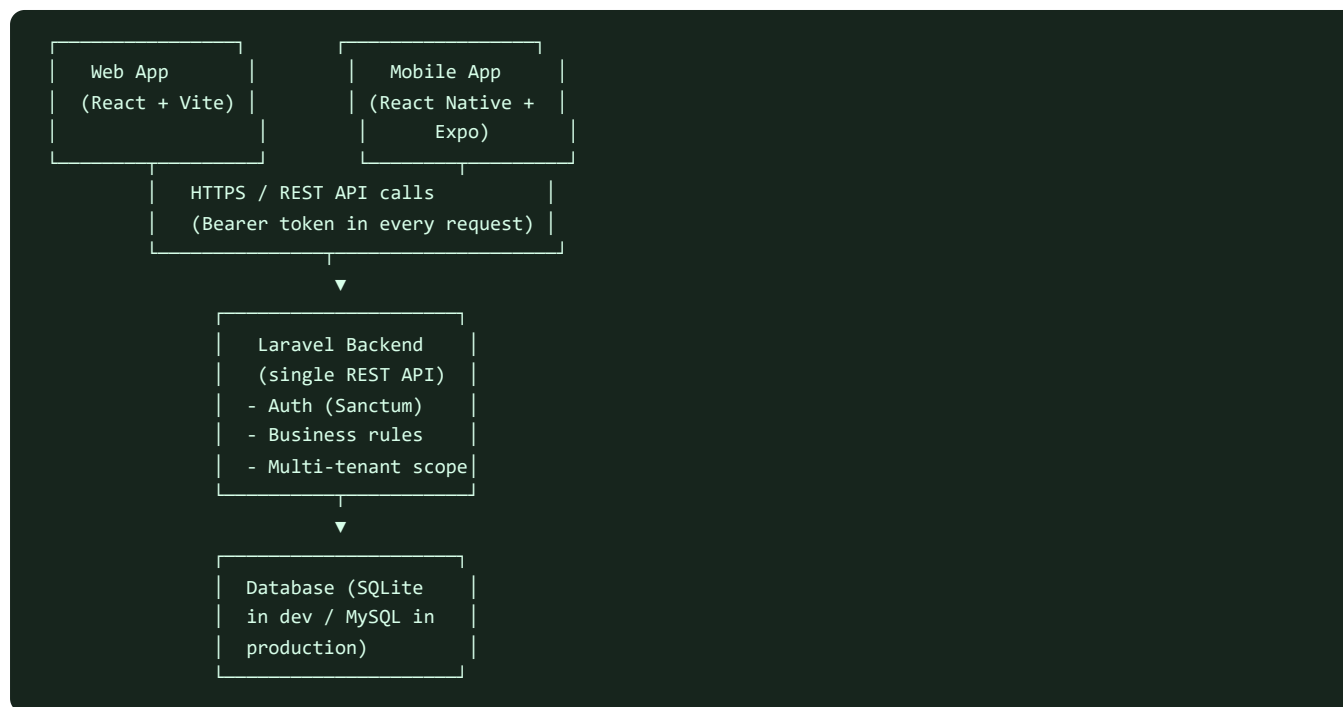
Technology	Version	Purpose
Expo (SDK)	~57.0	The framework/toolchain that runs and builds the React Native app
React Native	0.86	Lets the app be written once in JavaScript/React and run on Android and iOS
React Navigation (Drawer)	^7.13	The slide-out side menu navigation used per role
Expo Secure Store	~57.0	Safely stores the login token on the device
Expo Image Picker	~57.0	Lets a user choose a profile photo from their gallery
Expo Document Picker	~57.0	Lets the Owner choose an Excel/CSV file to import from the phone's storage
Expo Build Properties	~57.0	Lets Android connect to a local (non-HTTPS) backend during development/testing
EAS (Expo Application Services)	cloud service	Builds the installable APK / production app bundle in the cloud

2.4 Why these choices?

- **Laravel** was chosen because it comes with authentication, database handling (migrations), file uploads, and job scheduling already built in — much faster to build a solid backend than writing everything from scratch.
- **React + React Native** together mean the same team/skillset (JavaScript) builds both the web and mobile app, and a lot of logic (like how WhatsApp receipts are built, or how search works) is written almost identically in both places.
- **SQLite** is only for local development — see Section 9 for the production database plan (MySQL).

3. System Architecture

3.1 High-Level Flow



3.2 Multi-Tenancy (Many Businesses, One System)

Every important table (items, categories, orders, customers, purchases, staff, etc.) has a hidden `tenant_id` column. Every database query is automatically filtered to the logged-in user's own `tenant_id` — this is enforced at the model level (a Laravel "Global Scope"), so it is not something a developer can accidentally forget to add to a new query. This means:

- Business A can never see Business B's items, orders, or staff.
- The **Super Admin** account is the only role that is not tied to one tenant — it can see the list of all registered businesses and suspend/reactivate them, but it cannot see any business's actual store data (items, orders, customers).

3.3 Real-Time Order Updates (Web) vs. Polling (Mobile)

When a Counter Staff member places a new order, the assigned Store Worker's pick-list screen needs to show it immediately.

- **On the Web App:** Laravel Reverb (a WebSocket server) pushes the new order to the browser the instant it is created — no refresh or delay.
- **On the Mobile App:** instead of a WebSocket connection, the app checks for new orders automatically every 6 seconds. This was a deliberate simplicity choice — it works reliably on any network and needs no extra mobile-specific real-time setup.

3.4 Worker Auto-Assignment (Load Balancing)

When a new order is placed, the backend automatically decides which Store Worker should collect it:

1. Find all Store Workers who are currently **free** (no order currently assigned to them that is still "placed" or "picking").
2. Among the free workers, pick the one who has been assigned the **fewest orders overall** (fair rotation).
3. If **no worker is free**, the order is left unassigned ("queued") and is invisible to all workers.
4. As soon as any worker finishes their current order (marks it "ready"), the system automatically looks for the oldest queued order and assigns it to that now-free worker.

4. User Roles & Permissions

Role	Can do
Super Admin	Views the list of all registered businesses (tenants). Can suspend or reactivate a business's access. Can also permanently delete a business (see Section 5.4) — removes all its data and staff accounts and frees up the owner's email for reuse. Cannot see any business's items, orders, or staff.
Owner	Full control of their own business: Categories, Items (+ Excel import), New Order, Payments, Low Stock Alerts, Purchases (restocking), Reports & Analytics, Staff management, Staff Activity report, Settings. Can also do everything Counter Staff and Store Worker can do (oversight).
Counter Staff	Take new customer orders (search-and-add items), process Payments (cash/card/credit), send WhatsApp receipts, manage their own account Settings.
Store Worker	Sees only the orders assigned to them; marks each item as collected off the shelf; order becomes "Ready for payment" once all items are collected.

5. Complete Feature List

Features are grouped the way the project was scoped and built — **Base** (core day-to-day system), **Intermediate** (analytics/reporting layer), and **Advanced** (multi-business platform + messaging + mobile app).

5.1 Base Version

Feature	Details
Authentication	Login, self-service business registration (email verification code), Forgot/Reset Password.
Categories & Items	Full add/edit/delete for product categories and inventory items (price, purchase cost, stock quantity, low-stock threshold).
Excel/CSV Bulk Import	Owner uploads an Excel or CSV file with columns Category, Name, Price, Purchase Price, Stock Quantity, Low Stock Threshold — the system creates new items and updates existing ones automatically. A downloadable template is provided.
Order Taking (Search-based)	Instead of browsing categories one by one, staff type an item name or a category name (e.g. "juice") into a search box and matching items appear instantly below it.
Live Worker Pick-List	Store Worker sees their assigned order appear on screen in real time (web) or within 6 seconds (mobile), and taps items off as they are collected.
Automatic Worker Assignment	See Section 3.4 — orders are automatically routed to whichever worker is free, with automatic queuing and dispatch.
Payments	Cash, Card, or Credit (Due) payment types. Stock is automatically deducted when an order is placed.
WhatsApp Receipt	After payment, a "Send Receipt" button appears next to the customer's name — tapping it opens WhatsApp with the bill pre-filled, ready to send to the customer's number.
Purchases (Restocking)	Owner records new stock purchases from suppliers, which increases item stock quantities.
Low Stock Alerts	A dedicated page listing every item at or below its configured low-stock threshold.
Owner Dashboard	At-a-glance summary of the business (sales, order counts, etc.).
Staff Management	Owner creates/removes Counter Staff and Store Worker accounts for their own business.
Staff Activity Report	Shows how many orders each Counter Staff member has taken and each Store Worker has picked, plus total sales handled.
Settings	Profile photo upload, password change (with show/hide "eye" icon on both web and mobile), and on mobile — managing multiple saved server addresses.

5.2 Intermediate Version — Reports & Analytics (Done)

Feature	Details
Sales Trend Chart	Line chart of sales over time.
Top Items Chart	Bar chart of best-selling items.
Payment Type Breakdown	Donut chart of Cash vs. Card vs. Credit split.
CSV Export	Reports can be exported/downloaded as CSV files.

5.3 Advanced Version

Feature	Status	Details
Multi-Tenancy	Done	Many businesses on one system, each fully isolated. Super Admin panel to oversee all registered businesses.
Mobile App (React Native + Expo)	Done	Full feature parity with the web app for all four roles, installable as an Android APK.
Real SMS Due-Payment Reminders	Code complete — pending Twilio account setup	A scheduled command checks for overdue "credit" payments and sends a real SMS to the customer via Twilio. All the code is written and tested — it only needs a real Twilio account (SID, Auth Token, phone number) added to the server's <code>.env</code> file to start actually sending messages. See Section 10.

5.4 Recent Additions — Order Workflow & Admin Tools (Done)

Added after the base system was already live and in daily use, based on real usage feedback. All items below exist identically on both Web and Mobile unless noted otherwise.

Feature	Details
Dashboard Drill-Downs & Search	The "Today's Sales", "Today's Orders", and "Pending Dues" cards on the Owner dashboard are now real links — tapping one opens a dedicated, searchable page (filter by date range and by customer name/phone/order number). The dashboard itself now only shows the 2–3 most recent entries per section (instead of an ever-growing list), with a "View all" link to the full searchable page.
Order Attribution — Who Booked / Who Picked	The Owner's "Order History" page shows a Booked By column (which Counter Staff took each order) and can filter by a specific staff member. Counter Staff have their own "My Orders" page showing only orders they personally booked. Store Workers have their own "My Orders" page showing every order ever assigned to them, with a Total Picked / Completed Today summary.
Priority ("Urgent") Orders	An "Urgent" checkbox is available when placing an order (e.g. a walk-in customer who needs their order picked immediately). When a Store Worker becomes free, an urgent order is handed to them ahead of the rest of the waiting queue — but an urgent order never interrupts a worker who is already actively picking a different order.
Recheck / Confirm Ready	Previously, an order flipped to "Ready" automatically the instant the last item was ticked off — risking a rushed mix-up between two customers' orders. Now, once every item is ticked, the worker sees a final review list and must explicitly tap "Review & Confirm Ready" before the order moves to payment.
Editing an In-Progress Order	Owner/Counter Staff can now edit an order's items (add, remove, or change quantity) any time before it is marked Ready — e.g. the customer changes their mind mid-pick. The assigned worker's screen reflects the edit live (instantly on Web, within about 6 seconds on Mobile) without needing a refresh.
Mobile Keyboard Fix	Input fields on the Settings, New Order, and Payments screens no longer get hidden behind the on-screen keyboard while typing.
Collapsible Sidebar & Independent Scrolling (Web)	The web sidebar can be collapsed to icons-only via a toggle button (the choice is remembered between visits). The sidebar and the main content area now also scroll independently of each other, so a long page of content no longer drags the sidebar out of view.
Permanent Business Deletion (Super Admin)	Beyond Suspend/Reactivate, the Super Admin can now permanently delete a business — mainly meant for cleaning up test accounts created during development. This removes every item, category, order, customer, purchase, and due reminder belonging to that business, along with all of its staff accounts, which frees up the owner's email to be used again in a fresh registration. This action is irreversible, so it requires typing the exact business name to confirm before it proceeds.

6. Local Development Setup (Step by Step)

6.1 Requirements

- PHP 8.3+ with Composer installed
- Node.js (v18+) with npm
- Expo Go app (for quick testing) or Android Studio (for a full emulator) — optional if only testing on a real phone
- A code editor (VS Code)

6.2 Backend Setup

```
cd Backend
composer install
copy .env.example .env          (Windows) / cp .env.example .env  (Mac/Linux)
php artisan key:generate
php artisan migrate
php artisan serve
```

This starts the API at `http://127.0.0.1:8000`. For real-time order updates, also run Reverb in a second terminal:

```
php artisan reverb:start
```

6.3 Frontend (Web App) Setup

```
cd Frontend
npm install
npm run dev
```

This opens the web app at `http://localhost:5173` (Vite's default port). Make sure the Backend server (Section 6.2) is running at the same time.

6.4 Mobile App Setup

```
cd Mobile
npm install
npx expo start
```

Then scan the QR code shown in the terminal using the **Expo Go** app on your phone (for quick testing), or press `a` to open an Android emulator.

Important: the phone and the computer running the Backend must be on the same Wi-Fi network. Inside the Mobile App's **Settings** → **API Address** screen, enter the computer's local network IP address (e.g. `http://192.168.1.5:8000`) instead of `localhost` — a phone cannot reach "localhost" on the computer.

7. Complete Command Reference

This section explains *what* each command does. Section 13 (Appendix) lists every command by itself with no explanation, for quick copy-paste reference.

7.1 Backend (run inside the `Backend` folder)

Command	What it does
<code>composer install</code>	Downloads all PHP packages the backend needs
<code>php artisan key:generate</code>	Generates the app's secret encryption key (run once)
<code>php artisan migrate</code>	Creates/updates all database tables based on the migration files
<code>php artisan migrate:fresh</code>	Drops all tables and re-creates them from scratch (wipes all data — dev only)
<code>php artisan migrate:fresh --seed</code>	Same as above, then fills in demo/test data (seeded accounts)
<code>php artisan serve</code>	Starts the API server (default: <code>http://127.0.0.1:8000</code>)
<code>php artisan reverb:start</code>	Starts the real-time WebSocket server used for live order updates
<code>php artisan queue:listen</code>	Processes background jobs (e.g. sending SMS) — needed if any feature is queued
<code>php artisan reminders:send-due</code>	Manually runs the due-payment SMS reminder check (normally scheduled to run automatically once a day)
<code>php artisan tinker</code>	Opens an interactive PHP console connected to the app — used for quick manual data checks/fixes during development
<code>composer require <package></code>	Adds a new PHP package to the project (e.g. <code>maatwebsite/excel</code>)

7.2 Frontend / Web App (run inside the `Frontend` folder)

Command	What it does
<code>npm install</code>	Downloads all JavaScript packages the web app needs
<code>npm run dev</code>	Starts the local development server with live-reload
<code>npm run build</code>	Builds the optimized, production-ready static files (into a <code>dist</code> folder) — this is what gets uploaded to a hosting server
<code>npm run preview</code>	Serves the production build locally, to double check it before deploying
<code>npm run lint</code>	Checks the code for style/quality issues

7.3 Mobile App (run inside the `Mobile` folder)

Command	What it does
<code>npm install</code>	Downloads all JavaScript packages the mobile app needs
<code>npx expo start</code>	Starts the Expo development server — shows a QR code to open the app in Expo Go

<code>npx expo start --android</code>	Same, but tries to open directly in a connected Android emulator/device
<code>npx expo install <package></code>	Installs a new package, automatically choosing the version that matches this Expo SDK (safer than plain <code>npm install</code> for Expo packages)
<code>npx eas login</code>	Logs into your Expo/EAS account from the terminal (needed before building)
<code>eas build --profile preview --platform android</code>	Builds a real, installable Android APK file in the cloud (see Section 8)
<code>eas build --profile production --platform android</code>	Builds the final release version meant for the Google Play Store
<code>eas build --profile production --platform ios</code>	Builds the final release version meant for the Apple App Store
<code>eas submit</code>	Uploads a finished build directly to the Google Play / Apple App Store

8. Building & Distributing the Mobile App (EAS Build)

Because a phone app cannot simply be "opened in a browser" the way a website can, it must be **built** into an installable file. StoreFlow uses **EAS (Expo Application Services)** — Expo's own cloud build service — so no local Android Studio / Xcode installation is required to produce a real app file.

8.1 The Three Build Profiles (in `eas.json`)

Profile	Purpose	Output
<code>development</code>	A build with extra developer tools attached, for active coding/debugging	Development client app
<code>preview</code>	A real, shareable test build to install directly on a phone (no app store needed)	<code>.apk</code> file — directly installable
<code>production</code>	The final version meant to be published to the Play Store / App Store	<code>.aab</code> (Android App Bundle) — Play Store format

Why "preview" builds an APK and "production" builds an AAB: An AAB cannot be installed directly on a phone — it can only be uploaded to the Play Store, which then generates the actual installable file per device. An APK, on the other hand, can be shared and installed directly (perfect for testing with the client before an official store release).

8.2 Typical Build & Test Cycle

```
cd Mobile
npx eas login
eas build --profile preview --platform android
```

Wait for the build to finish in the cloud (a link is printed in the terminal, and it's also visible on expo.dev). Download the `.apk` file to the phone and install it.

App icon / name changes require a new build. Editing `app.json` or replacing an icon image has *no effect* on an already-installed app — a new `eas build` must be run, and the old app must be fully **uninstalled** (not just overwritten) before installing the new one, since some Android launchers cache the old icon/name otherwise.

8.3 Key `app.json` Settings

Field	Purpose
<code>name</code>	The app name shown under the icon and during install
<code>slug</code>	A URL-safe internal identifier for the Expo project
<code>icon</code>	The app icon shown on iOS and older Android — must be a square PNG (1024×1024 recommended)
<code>android.package</code>	The unique Android package ID (e.g. <code>com.astutemint.storeflow</code>) — cannot be changed once published to the Play Store
<code>android.adaptiveIcon</code>	The modern Android icon system (foreground image + background color/image) — takes priority over the plain <code>icon</code> field on Android 8+

9. Hosting & Deployment Plan

Not yet implemented — documented here as the agreed plan for when the project is ready to go live.

9.1 Backend (Laravel API) Hosting

Option	Notes
VPS (e.g. DigitalOcean, Hostinger VPS, Contabo)	Full control server. Needs manual setup of PHP, Composer, Nginx/Apache, and a process manager. Best long-term option, most flexible.
Managed Laravel hosting (e.g. Laravel Forge + any VPS provider)	Forge handles server setup, SSL, deployments automatically — recommended if avoiding manual server administration.
Shared hosting with PHP/Composer support	Cheapest option, but many shared hosts don't allow the background processes StoreFlow needs (queue worker, Reverb) — needs to be checked with the host first.

9.2 Database — Move from SQLite to MySQL

SQLite (a single file) is fine for development but is not recommended for a live, multi-tenant production system. The plan is to switch `DB_CONNECTION` in `.env` to `mysql` and provide real database host/username/password — Laravel's migrations run unchanged on MySQL, no application code changes needed.

9.3 Frontend (Web App) Hosting

- Run `npm run build` to produce a static `dist` folder.
- Host it either on the same server as the backend (served by Nginx/Apache as static files) or on a static-hosting service like Vercel or Netlify (free tier available, simplest option, automatic SSL/CDN).
- The web app just needs one setting: the backend API's live URL.

9.4 Real-Time (Reverb) in Production

Laravel Reverb needs a long-running process (not just a normal web request) to keep WebSocket connections open. On a VPS this is managed with a process supervisor (e.g. Supervisor) running `php artisan reverb:start` permanently in the background. As a simpler alternative, a hosted WebSocket service (e.g. Pusher's cloud service, which Laravel already supports through the same Echo/Pusher-js libraries used in the frontend) can be used instead of self-hosting Reverb.

9.5 Domain & SSL

- A domain name is pointed at the server (e.g. `api.storeflow.com` for the backend, `app.storeflow.com` for the web app).
- Free SSL certificates (Let's Encrypt) are used so everything runs on `https://` — required for the mobile app to connect reliably and for browsers to not show a security warning.

9.6 Mobile App Store Publishing

Store	Notes
Google Play Store	One-time \$25 developer account fee. Can start with "Internal Testing" (instant, small group) before a full public release.
Apple App Store	\$99/year developer account fee. Requires an Apple Developer account and review process (typically 1–3 days).

10. SMS & WhatsApp Messaging Solution

10.1 WhatsApp Receipts — Live Now

After a payment is processed, a green "Send Receipt" WhatsApp button appears next to the customer's name (on both web and mobile). Tapping it opens WhatsApp with the customer's number and a pre-written receipt message already filled in — the staff member just presses send.

Important limitation: this uses WhatsApp's free "click-to-chat" link (`wa.me`) — there is no way to check in advance whether a phone number is actually registered on WhatsApp. If it isn't, WhatsApp itself will show an error when the message is opened. There is no cost for this method, but it requires WhatsApp to be installed on the device sending the message, and a person still has to press "Send".

10.2 SMS Due-Payment Reminders — Code Complete, Needs Twilio Account

For customers who bought on credit ("due" payment) and haven't paid yet, StoreFlow can automatically send a real SMS reminder. This is fully coded and tested using Twilio, a paid SMS-sending service — it only needs a live Twilio account to actually start sending messages.

What is needed to go live:

1. Create an account at [twilio.com](https://www.twilio.com) and add billing (SMS costs a small fee per message, varies by country).
2. Get a Twilio phone number that can send SMS to the target country.
3. Copy three values from the Twilio dashboard: **Account SID**, **Auth Token**, and the **Twilio phone number**.
4. Add them to the Backend's `.env` file:

```
TWILIO_SID=your-account-sid
TWILIO_AUTH_TOKEN=your-auth-token
TWILIO_FROM_NUMBER=+1xxxxxxxxxx
TWILIO_DEFAULT_COUNTRY_CODE=+92
```

5. Run `php artisan reminders:send-due` once manually to confirm a real SMS arrives, then it runs automatically once a day on its own schedule going forward.

10.3 Future Option — Official WhatsApp Business API

If more reliable, fully-automated WhatsApp sending is wanted later (instead of the manual click-to-send method above), the official **WhatsApp Business Platform (Meta Cloud API)** is the upgrade path. It requires a Meta Business verification process and has its own per-message cost, but allows messages to be sent completely automatically, without a staff member needing to press "Send" in the WhatsApp app. This is **not yet built** — noted here purely as a possible future upgrade.

11. Security Notes

- Passwords are never stored in plain text — Laravel hashes them automatically.
- Login uses secure Bearer tokens (Sanctum) rather than storing a password on the device — a token can be revoked (logout) without changing the password.
- Every business's data is isolated at the database-query level (Section 3.2) — this is enforced centrally, not something that can be forgotten in one particular screen.
- Sensitive keys (Twilio credentials, Reverb secret, database password) live only in the server's `.env` file, which is never committed to source control.
- Before going live: change all demo/seeded passwords, generate a fresh `APP_KEY` for production, and set `APP_DEBUG=false` so detailed error messages are never shown to real users.
- The Super Admin's "permanently delete a business" action (Section 5.4) is irreversible and cannot be undone — it requires typing the exact business name to confirm, specifically to prevent an accidental click from wiping a real client's data.

12. Future Roadmap

Item	Status
Add real Twilio credentials and confirm SMS reminders send successfully	Pending client action
Move production database from SQLite to MySQL	Planned
Deploy Backend + Web App to a live server with a domain and SSL	Planned
Publish Mobile App to Google Play Store (and Apple App Store if needed)	Planned
(Optional, later) Upgrade WhatsApp receipts to the official WhatsApp Business API for full automation	Future consideration

13. Appendix — Full Terminal Command List

Every command used in this project, grouped by folder, with no explanation — for quick copy-paste reference while revising.

Backend/ folder

```
composer install
composer require maatwebsite/excel
composer require twilio/sdk
copy .env.example .env
php artisan key:generate
php artisan migrate
php artisan migrate:fresh
php artisan migrate:fresh --seed
php artisan serve
php artisan reverb:start
php artisan queue:listen --tries=1 --timeout=0
php artisan reminders:send-due
php artisan tinker
php artisan route:list
php artisan config:clear
php artisan cache:clear
```

Frontend/ folder

```
npm install
npm run dev
npm run build
npm run preview
npm run lint
```

Mobile/ folder

```
npm install
npx expo install expo-document-picker
npx expo install <any-other-expo-package>
npx expo start
npx expo start --android
npx expo start --ios
npx expo start --web
npx eas login
npx eas whoami
eas build --profile development --platform android
eas build --profile preview --platform android
eas build --profile production --platform android
eas build --profile production --platform ios
eas submit --platform android
eas submit --platform ios
```

Finding your computer's local network IP (for Mobile App → Backend connection)

```
# Windows (PowerShell / cmd)
ipconfig

# Mac / Linux
ifconfig
# or
hostname -I
```

Look for the "IPv4 Address" under your active Wi-Fi adapter — this is what gets typed into the Mobile App's Settings → API Address screen, e.g. `http://192.168.1.5:8000` .

StoreFlow — Project Documentation · Generated for internal, supervisor, and client reference